

1. Zastosowanie

Procedura Choleskiego rozwiązuje układ równań liniowych $A \cdot x = b$ z macierzą symetryczną dodatnio określoną metodą rozkładu Choleskiego w zmiennopozycyjnej arytmetyce przedziałowej.

2. Opis metody

Metoda Choleskiego pozwala na rozwiązanie układu równań $A \cdot x = b$ poprzez rozkład macierzy symetrycznej i dodatnio określonej A na iloczyn dolnej macierzy trójkątnej L i jej transpozycji L^T :

$$A = L \cdot L^T$$

Elementy macierzy L są obliczane według wzorów:

$$L_{j,j} = \sqrt{A_{j,j} - \sum_{k=1}^{j-1} L_{j,k}^2}$$
$$L_{i,j} = \frac{A_{i,j} - \sum_{k=1}^{j-1} L_{i,k} \cdot L_{j,k}}{L_{j,j}} \quad \text{dla } i > j$$

Następnie rozwiązanie układu sprowadza się do dwóch prostych etapów:

1. Rozwiązanie układu trójkątnego dolnego (podstawienie w przód): $L \cdot y = b$
2. Rozwiązanie układu trójkątnego górnego (podstawienie wstecz): $L^T \cdot x = y$

Wszystkie operacje arytmetyczne na liczbach zmiennoprzecinkowych są w tej procedurze zastąpione gwarantowanymi operacjami na przedziałach.

3. Wywołanie funkcji

`IntervalSolver::solve(input, mode);`

4. Dane

- `input` – obiekt struktury `ParsedInput` zawierający wejściowe reprezentacje tekstowe danych układu:
 - `input.A` – dwuwymiarowy kontener (`std::vector<std::vector<QString>>`) przechowujący elementy macierzy układu w postaci napisów. Rozmiar wewnętrzny determinuje liczbę równań n .

- input.b – jednowymiarowy kontener (`std::vector<QString>`) przechowujący elementy wektora wyrazów wolnych jako napisy.
- mode – parametr typu enum `IntervalMode` określający tryb weryfikacji danych:
 - `IntervalMode::RealInput` (wartość 0) – tryb wymagający, aby dane wejściowe reprezentowały wyłącznie liczby rzeczywiste (szerokość przedziałów wejściowych nie może przekraczać zdefiniowanej tolerancji $\epsilon = 10^{-14}$).
 - `IntervalMode::IntervalInput` (wartość 1) – tryb dopuszczający dowolne przedziały szerokie jako dane wejściowe.

5. Wynik

Funkcja zwraca strukturę `IntervalResult`. W przypadku poprawnego zakończenia obliczeń (`ok == true`), wyniki są formatowane tekstowo do pól:

- left – napis zawierający listę lewych końców przedziałów wektora rozwiązania x w formacie "[x_0.a, x_1.a, ...]".
- right – napis zawierający listę prawych końców przedziałów wektora rozwiązania x w formacie "[x_0.b, x_1.b, ...]".
- width – napis zawierający obliczone za pomocą funkcji `IntWidth` szerokości przedziałów dla poszczególnych składowych rozwiązania.

6. Inne parametry

Struktura wynikowa `IntervalResult` przechowuje dodatkowe parametry kontrolne i diagnostyczne:

- ok – zmienna typu `bool`; przyjmuje wartość `true`, jeśli układ został rozwiązany pomyślnie, lub `false`, jeśli wystąpił jakikolwiek błąd uniemożliwiający dalsze obliczenia.
- error – napis typu `std::string` zawierający szczegółowy opis zaistniałego błędu. Wartość ta jest ustawiana, gdy:
 - wymiary macierzy A i wektora b są niezgodne lub równe zero,
 - macierz A nie jest kwadratowa,
 - parsowanie napisów wejściowych się nie powiedzie (np. brak domknięcia nawiasu przedziału),

- o w trybie RealInput wykryte zostaną przedziały o szerokości większej niż 10^{-14} ,
 - o macierz A nie spełnia warunku symetrii w granicach tolerancji błędu,
 - o podczas rozkładu Choleskiego wystąpi próba pierwiastkowania wartości ujemnej (macierz nie jest dodatnio określona) lub dzielenia przez przedział zawierający zero.
- log – kontener `std::vector<std::string>` gromadzący tekstowy zapis operacji krok po kroku. Zapisywane są w nim stany pośrednie elementów macierzy trójkątnej L oraz wektorów y i x .

7. Typy parametrów

- ParsedInput – struktura danych wejściowych (zdefiniowana w zewnętrznym module `core/Parser.h`).
- IntervalMode – typ wyliczeniowy (enum class) dla trybu pracy algorytmu.
- IntervalResult – struktura przechowująca kompletny stan wyjściowy algorytmu.

8. Identyfikatory nielokalne

- Interval<long double> – klasa szablonowa przedziału zaimplementowana w nagłówku `Interval.h`.
- PINT_MODE – stała określająca tryb arytmetyki przedziałowej ustawiana metodą `Interval<T>::SetMode`.
- ISub, IMul, IDiv, ISqrt – funkcje realizujące podstawowe operacje arytmetyczne oraz pierwiastkowanie na obiektach przedziałowych.
- IntWidth – funkcja obliczająca szerokość przedziału zmiennej wyjściowej.
- QString – klasa łańcuchów znakowych frameworku Qt, reprezentująca wejściowe tokeny danych.

9. Tekst funkcji

```
IntervalResult IntervalSolver::solve(const ParsedInput &input, IntervalMode
mode) {
    IntervalResult result;
    try {
        const size_t n = input.A.size();
        if (n == 0 || input.b.size() != n) {
            result.ok = false;
            result.error = "Niepoprawne rozmiary A lub b.";
            return result;
        }

        Interval<long double>::Initialize();
        Interval<long double>::SetMode(PINT_MODE);

        std::vector<std::vector<Interval<long double>>>> A(n,
std::vector<Interval<long double>>(n));
        std::vector<Interval<long double>> b(n);

        for (size_t i = 0; i < n; ++i) {
            if (input.A[i].size() != n) {
                result.ok = false;
                result.error = "Macierz A musi być kwadratowa.";
                return result;
            }
            for (size_t j = 0; j < n; ++j) {
                Interval<long double> cell;
                if (!parseIntervalToken(input.A[i][j], cell, result.error)) {
                    result.ok = false;
                    return result;
                }
                A[i][j] = cell;
            }
        }

        for (size_t i = 0; i < n; ++i) {
            Interval<long double> cell;
            if (!parseIntervalToken(input.b[i], cell, result.error)) {
                result.ok = false;
                return result;
            }
            b[i] = cell;
        }

        if (mode == IntervalMode::RealInput) {
            const long double eps = 1e-14;
            for (size_t i = 0; i < n; ++i) {
                for (size_t j = 0; j < n; ++j) {
                    if (std::abs(A[i][j].a - A[i][j].b) > eps) {
                        result.ok = false;
                        result.error = "W trybie „przedziałowa dla danych
rzeczywistych” dozwolone są tylko liczby rzeczywiste (przedziały punktowe)!";
                        return result;
                    }
                }
            }
        }
    }
}
```

```

    }
    for (size_t i = 0; i < n; ++i) {
        if (std::abs(b[i].a - b[i].b) > eps) {
            result.ok = false;
            result.error = "W trybie „przedziałowa dla danych
rzeczywistych” dozwolone są tylko liczby rzeczywiste (przedziały punktowe)!";
            return result;
        }
    }
}

const long double eps = 1e-14;
for (size_t i = 0; i < n; ++i) {
    for (size_t j = i + 1; j < n; ++j) {
        if (fabsl(A[i][j].a - A[j][i].a) > eps || fabsl(A[i][j].b -
A[j][i].b) > eps) {
            result.ok = false;
            result.error = "Macierz A nie jest symetryczna.";
            return result;
        }
    }
}

std::vector<std::vector<Interval<long double>>> L(n,
std::vector<Interval<long double>>(n, Interval<long double>(0, 0)));
int steps = 0;

for (size_t i = 0; i < n; ++i) {
    for (size_t j = 0; j <= i; ++j) {
        Interval<long double> sum = A[i][j];
        for (size_t k = 0; k < j; ++k) {
            sum = ISub(sum, IMul(L[i][k], L[j][k]));
        }

        if (i == j) {
            int st = 0;
            Interval<long double> root = ISqrt(sum, st);
            if (st != 0) {
                result.ok = false;
                result.error = "Nie można wyznaczyć pierwiastka
(ujemny przedział).";
                return result;
            }
            L[i][j] = root;
        } else {
            L[i][j] = IDiv(sum, L[j][j]);
        }

        std::string left, right;
        L[i][j].IEndsToStrings(left, right);
        result.log.push_back("L[" + std::to_string(i) + "," +
std::to_string(j) + "] = [" + left + "," + right + "]");
    }
}

std::vector<Interval<long double>> y(n, Interval<long double>(0, 0));
for (size_t i = 0; i < n; ++i) {

```

```

        Interval<long double> sum = b[i];
        for (size_t k = 0; k < i; ++k) {
            sum = ISub(sum, IMul(L[i][k], y[k]));
        }
        y[i] = IDiv(sum, L[i][i]);
        std::string left, right;
        y[i].IEndsToStrings(left, right);
        result.log.push_back("y[" + std::to_string(i) + "] = [" + left +
", " + right + "]"");
    }

    std::vector<Interval<long double>> x(n, Interval<long double>(0, 0));
    for (int i = static_cast<int>(n) - 1; i >= 0; --i) {

        Interval<long double> sum = y[i];
        for (size_t k = i + 1; k < n; ++k) {
            sum = ISub(sum, IMul(L[k][i], x[k]));
        }
        x[i] = IDiv(sum, L[i][i]);
        std::string left, right;
        x[i].IEndsToStrings(left, right);
        result.log.push_back("x[" + std::to_string(i) + "] = [" + left +
", " + right + "]"");
    }

    std::ostringstream leftOut;
    std::ostringstream rightOut;
    std::ostringstream widthOut;
    leftOut << "[";
    rightOut << "[";
    widthOut << "[";
    for (size_t i = 0; i < n; ++i) {
        std::string left, right;
        x[i].IEndsToStrings(left, right);
        long double width = IntWidth(x[i]);

        if (i > 0) {
            leftOut << ", ";
            rightOut << ", ";
            widthOut << ", ";
        }
        leftOut << left;
        rightOut << right;
        widthOut << formatLongDouble(width);
    }
    leftOut << "]"";
    rightOut << "]"";
    widthOut << "]"";

    result.ok = true;
    result.left = leftOut.str();
    result.right = rightOut.str();
    result.width = widthOut.str();
    return result;
} catch (const std::exception &ex) {
    result.ok = false;

```

```

        result.error = std::string("Błąd w arytmetyce przedziałowej: ") +
ex.what();
        return result;
    } catch (...) {
        result.ok = false;
        result.error = "Błąd w arytmetyce przedziałowej: nieznany wyjątek.";
        return result;
    }
}

```

10. Przykłady

a) Przykład 1: Dane punktowe

- **Dane wejściowe:**
 - $A = [[3, 1, 0], [1, 2, -1], [0, -1, 3]]$
 - $b = [0, 1, 0]$
- **Wyniki operacji:**
 - $\text{result.left} = [-2.5000000000000001E-1, 7.499999999999999E-1, 2.499999999999999E-1]$
 - $\text{result.right} = [-2.499999999999999E-1, 7.5000000000000001E-1, 2.5000000000000001E-1]$
 - $\text{result.width} = [0.000000e+00, 0.000000e+00, 0.000000e+00]$

b) Przykład 2: Dane przedziałowe

- **Dane wejściowe:**
 - $A = [[3, 1, 0], [1, 2, -1], [0, -1, 3]]$
 - $b = [0, [0.99, 1], 0]$
- **Wyniki operacji:**
 - $\text{result.left} = [-2.5000000000000001E-1, 7.424999999999999E-1, 2.474999999999999E-1]$
 - $\text{result.right} = [-2.474999999999999E-1, 7.5000000000000001E-1, 2.5000000000000001E-1]$
 - $\text{result.width} = [2.500000e-03, 7.500000e-03, 2.500000e-03]$

c) Przykład 3: Dane błędne (Macierz niesymetryczna)

- **Dane wejściowe:**

- $A = \begin{bmatrix} [1.0, 1.1], [2.0, 2.1], [3.0, 3.1] \\ [2.0, 2.1], [4.0, 4.1], [5.0, 5.1] \\ [3.5, 3.6], [5.0, 5.1], [6.0, 6.1] \end{bmatrix}$
- $b = [1, 1.1], [2, 2.1], [3, 3.1]$

- **Wyniki operacji:**

- `result.error = "Macierz A nie jest symetryczna."`